

Algorithms for Data Science

Lecture #8: Graphs, Networks, and Spatial Search

Mateusz Buczynski

University of Warsaw

13.05.2026

Thanks to dr Krzysztof Fleszar for the base material.

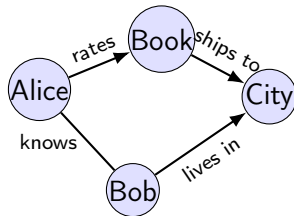
Why Graphs Matter

Graphs model relationships

Vertices represent objects, and edges represent interactions, transitions, or dependencies.

Examples

Road networks, social networks, recommendation systems, molecules, web links, game states.



Basic Vocabulary

Notation

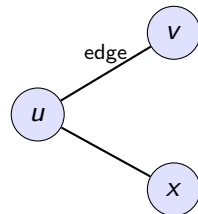
$$G = (V, E)$$

or, for weighted graphs,

$$G = (V, E, w).$$

Edge

A connection between two vertices.



a graph is a set of vertices and edges

Vertex

A node in the graph.

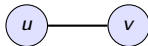
Parallel

Graphs can have multiple parallel connections.

Directed, Undirected, Weighted

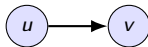
Undirected

Friendship, symmetric relation.



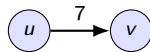
Directed

Following, one-way streets.



Weighted

Travel time, cost, distance.



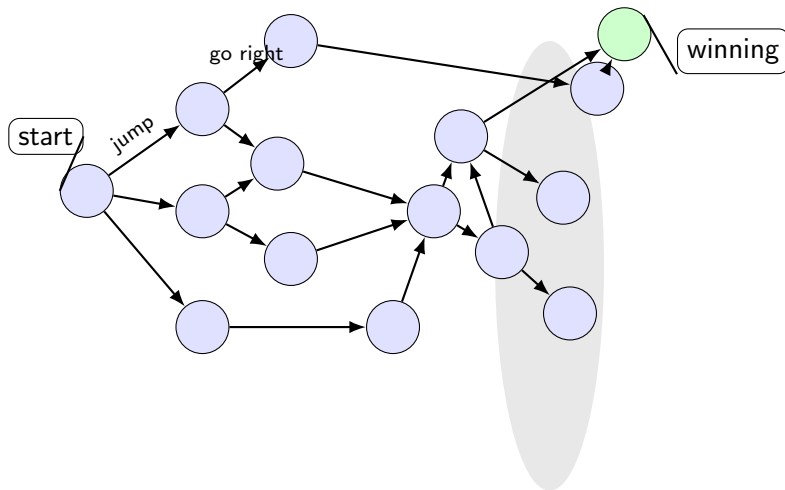
Modeling matters

The right graph model is already half of the algorithmic solution.

Example: Video Game

```
state = (position, time, gold)
```

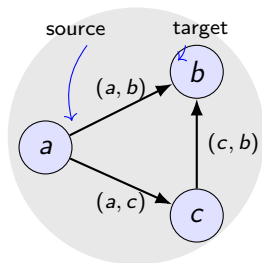
vertices = states



How to Represent a Graph?

Conventions

n = number of vertices, m = number of edges



Same graph, different data structures

We can store the same edges as:

- an adjacency matrix
- adjacency lists
- an edge list

	a	b	c
a	0	1	1
b	0	0	0
c	0	1	0

$a : b \rightarrow c \rightarrow \text{None}$

Adjacency Matrix vs Adjacency Lists

Adjacency matrix

$\Theta(n^2)$ space

Fast edge-existence check.

	<i>a</i>	<i>b</i>	<i>c</i>
<i>a</i>	0	1	1
<i>b</i>	1	0	0
<i>c</i>	1	0	0

Adjacency lists

$\Theta(n + m)$ space

Usually better for sparse graphs.

a : *b*, *c*
b : *a*
c : *a*

Default choice

For algorithmic work on sparse graphs, adjacency lists are usually the right representation.

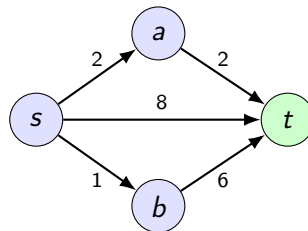
Shortest vs Cheapest Paths

Shortest path

Minimize the number of edges.

Cheapest path

Minimize total edge weight.



fewest edges: $s \rightarrow t$ (cost 8)

cheapest: $s \rightarrow a \rightarrow t$ (cost 4)

Key warning

These are the same only when all edges have equal weight.

What Could the Edge Mean?

Scenario

You want to build a route-planning tool for a delivery company.

What Could the Edge Mean?

Scenario

You want to build a route-planning tool for a delivery company.

What should an edge weight represent?

- physical distance
- travel time
- fuel cost
- tolls
- expected delay under traffic

Breadth-First Search (BFS): Goal

Input

An unweighted graph and a start vertex s .

Output

For every reachable vertex v , compute the length of the shortest path from s to v in number of edges.

Why BFS Uses a Queue

Core idea

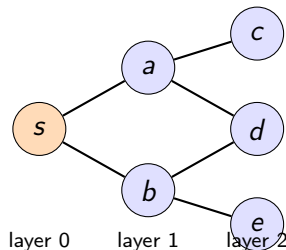
Vertices must be processed in order of discovery time.

Queue effect

First discovered, first expanded. This creates layers:

$0, 1, 2, 3, \dots$

from the source.



BFS Algorithm Template

Data

Queue Q , distance array $dist$, optional parent array $parent$.

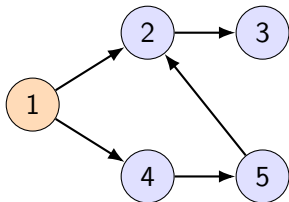
Flow

- 1 initialize all distances to $+\infty$
- 2 set $dist[s] = 0$, push s into Q
- 3 while Q is non-empty, pop the front vertex
- 4 relax all still-unvisited neighbors

Breadth-First Search (BFS): Visual View

Goal

For each vertex w , compute the shortest s - w path in number of edges.



vertex	1	2	3	4	5
dist	0	1	2	1	2

Queue snapshot during the run

$[1] \rightarrow [2, 4] \rightarrow [4, 3] \rightarrow [3, 5] \rightarrow [5]$

Algorithm sketch

```

dist[s] = 0
enqueue(s)
while Q not empty do
  v = extractOldest()
  for each edge (v,w) do
    if dist[w] not set then
      dist[w] = dist[v] + 1
    enqueue(w)
  
```

FIFO queue $\Rightarrow$ process vertices layer by layer.

BFS Complexity

With adjacency lists

Each vertex is enqueued at most once and each edge is scanned at most once.

Running time

$$\Theta(n + m)$$

Use it when

The graph is unweighted or all edge costs are identical.

Depth-First Search (DFS): Goal

Different behavior, different objective

DFS explores one branch deeply before backtracking.

Typical uses

Connectivity, topological ideas, cycle detection, component discovery, search-space exploration.

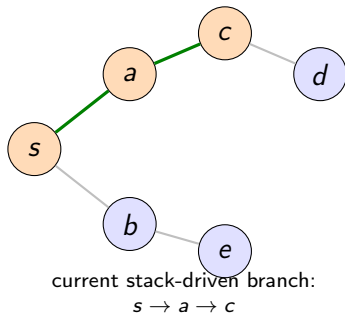
Why DFS Uses a Stack

Core idea

The most recently discovered unexplored vertex should be processed next.

Stack effect

Go deep first, then come back.



DFS Algorithm Template

Iterative version

Keep an explicit stack of vertices to process.

Flow

- 1 push s
- 2 pop a vertex
- 3 if not visited, mark it and push its neighbors

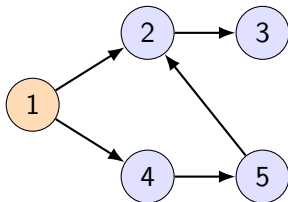
Equivalent recursive view

The call stack plays the role of the explicit stack.

Depth-First Search (DFS): Visual View

Goal

Explore one branch as deeply as possible before backtracking.



vertex	1	2	3	4	5
visit order	1	2	3	4	5

Stack snapshot during the run

$[1] \rightarrow [4, 2] \rightarrow [4, 3] \rightarrow [4] \rightarrow [5]$

Algorithm sketch

```

push(s)
while S not empty do
  v = pop()
  if v not visited then
    mark v visited
    for each edge (v,w) do
      push(w)
  
```

LIFO stack $\Rightarrow$ keep following the newest branch.

DFS Complexity

With adjacency lists

Like BFS, DFS visits every vertex and scans every edge at most once.

Running time

$$\Theta(n + m)$$

BFS vs DFS

BFS

- queue
- layer by layer
- shortest paths in unweighted graphs

DFS

- stack or recursion
- go deep first
- better for exploration and structure

Worked Example: BFS by Hand

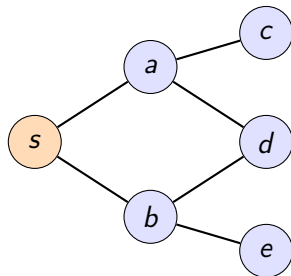
Task

Start at vertex s . In what order does BFS discover the vertices?

Assume alphabetical neighbor order

Track three things:

- queue contents
- discovery order
- distance labels



Instructor use

Good point to pause for 3–5 minutes and have students simulate the queue.

Worked Example: DFS by Hand

Task

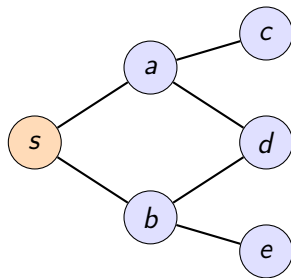
Run DFS from s on the same graph.

Ask

How does the order change if neighbors are processed alphabetically?

Compare with BFS

Same graph, same source, very different exploration pattern.



Checkpoint: BFS or DFS?

Question 1

You need shortest paths in an unweighted graph. Which traversal is the right default?

Question 2

You need to explore a search tree deeply and backtrack naturally. Which traversal fits better?

Question 3

How much can the output order depend on the order of neighbors in the adjacency lists?

Dijkstra's Algorithm: Problem Setup

Input

A weighted graph with nonnegative edge weights and a source s .

Goal

Compute the cheapest path from s to every vertex.

From BFS to Dijkstra

What changes?

In BFS, the next vertex to expand is the oldest in the queue.

In Dijkstra

The next vertex should be the currently cheapest not-yet-finalized vertex.

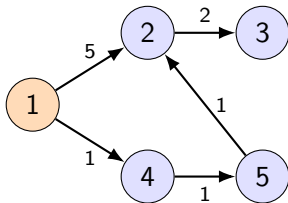
Data structure shift

Queue $\rightarrow$ priority queue.

Dijkstra's Algorithm: Visual View

Goal

For each vertex w , compute the cheapest s - w path.



vertex	1	2	3	4	5
dist	0	3	5	1	2

Priority queue snapshot during the run

$(1, 0) \rightarrow [(4, 1), (2, 5)] \rightarrow [(5, 2), (2, 5)]$
 $\rightarrow [(2, 3)] \rightarrow [(3, 5)]$

Algorithm sketch

```

dist[s] = 0
add(s, 0)
while PQ not empty do
  v = extractMin()
  for each edge (v, w) do
    if dist[w] > dist[v] + c(v, w) then
      dist[w] = dist[v] + c(v, w); add(w, dist[w])
  
```

Priority queue $\Rightarrow$ always expand the cheapest candidate next.

Dijkstra: Relaxation Rule

At an edge (u, v) of cost $w(u, v)$

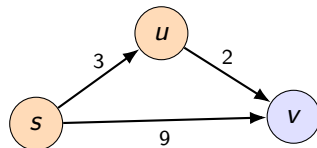
If going through u improves the best known cost to v , update it:

$$\text{dist}[v] > \text{dist}[u] + w(u, v)$$

$$\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$$

Optional extra data

Store $\text{parent}[v] = u$ to reconstruct the actual path.



$\text{dist}[u] = 3$
old $\text{dist}[v] = 9$
new via u : $3 + 2 = 5$

Why Dijkstra Works

Key invariant

When a vertex is extracted from the priority queue as the cheapest candidate, its distance is already final.

Why nonnegative weights matter

No later path can sneak in and become cheaper through a negative edge.

Dijkstra Complexity

Adjacency lists + binary heap

$$O((n + m) \log n)$$

Interpretation

Good for sparse weighted graphs.

Recovering Actual Paths

Parent array idea

Whenever we improve $dist[v]$, set

$$parent[v] = u.$$

To reconstruct path to t

Follow

$$t, parent[t], parent[parent[t]], \dots$$

back to the source, then reverse the sequence.

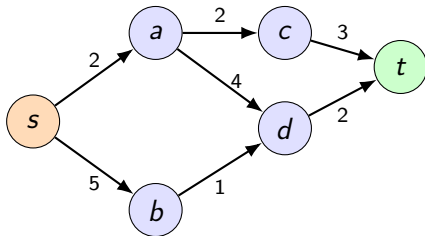
Worked Example: Dijkstra Trace

Task

Start from s . Which vertex is finalized next after each extraction?

Track

- current priority queue minimum
- tentative distance table
- parent pointers



Single-Source vs All-Pairs

Single-source

One source s , distances to all vertices.

All-pairs

Distances between every ordered pair (u, v) .

Typical question

If many source-target pairs will be queried, all-pairs preprocessing can be worth it.

Floyd-Warshall: Goal

Input

A weighted directed graph.

Output

An $n \times n$ matrix of shortest-path distances between all vertex pairs.

Floyd-Warshall: DP State

State meaning

After stage k ,

$$\delta[v][w]$$

is the cheapest cost from v to w using only internal vertices from

$$\{1, \dots, k\}.$$

Floyd-Warshall: Initialization

Base case $k = 0$

$$\delta[v][w] = \begin{cases} 0, & v = w, \\ w(v, w), & \text{if edge } (v, w) \text{ exists,} \\ \infty, & \text{otherwise.} \end{cases}$$

Interpretation

With no allowed internal vertices, only direct edges and trivial paths remain.

Floyd-Warshall: Update Rule

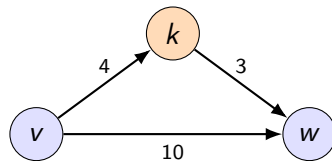
At stage k

For every pair (v, w) , compare

$$\delta[v][w]$$

with the path that goes through k :

$$\delta[v][k] + \delta[k][w].$$



direct: 10
through k : $4 + 3 = 7$

Transition

$$\delta[v][w] \leftarrow \min(\delta[v][w], \delta[v][k] + \delta[k][w]).$$

Floyd-Warshall: Why the DP Works

Decision at stage k

The best path from v to w :

- either does not use vertex k internally
- or does use k internally

This is classical DP

The recurrence splits the answer into two smaller, already solved possibilities.

Floyd-Warshall Complexity

Time

$$\Theta(n^3)$$

Space

$$\Theta(n^2)$$

Use it when

The graph is dense, the vertex set is not too large, or all-pairs distances are needed.

Worked Example: One Floyd-Warshall Update

Mini-instance

Suppose we currently know

$$\delta[a][d] = 12, \quad \delta[a][c] = 5, \quad \delta[c][d] = 4.$$

At stage $k = c$, what should happen to $\delta[a][d]$?

Discussion

Compare the old direct best value 12 with the route through c :

$$5 + 4 = 9.$$

So the entry should be improved to 9.

Why this helps pedagogically

Students often understand Floyd-Warshall better from one table update than from the

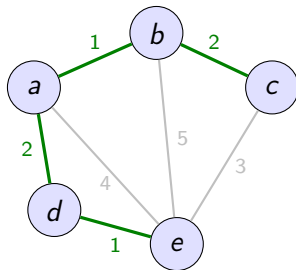
Minimum Spanning Tree (MST)

Input

A connected, weighted, undirected graph.

Goal

Choose a subset of edges that connects all vertices with minimum total weight and no cycles.



Kruskal's Algorithm

Greedy rule

Sort edges by weight from smallest to largest, then add an edge if it does not create a cycle.

Cycle test tool

Use union-find to check whether the endpoints are already in the same component.

Why Kruskal Works

Intuition

Whenever we connect two currently separate components with the cheapest possible edge, we never regret that choice.

Key property

The algorithm grows a forest that gradually merges into one spanning tree.

Kruskal Complexity

Dominant cost

Sorting the edges:

$$O(m \log m)$$

With union-find

Connectivity checks are nearly constant-time in practice.

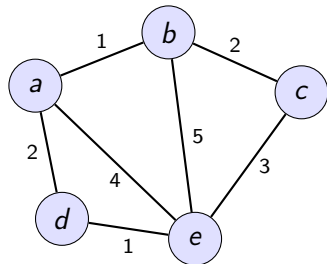
Worked Example: Kruskal by Hand

Task

Process edges in increasing order of weight and decide which ones enter the MST.

Class prompt

At which step is an edge rejected because it would create a cycle?



Break Point

Part 1 and Part 2 complete

Suggested break: 10–15 minutes

Before the break

Students should now be able to choose between BFS, DFS, Dijkstra, Floyd-Warshall, and Kruskal for standard path and connectivity tasks.

Graphs as Networks

Beyond paths

In data science, graphs often represent influence, importance, traffic, or diffusion.

Typical question

Which nodes are central, influential, or structurally important?

PageRank: Core Intuition

Web interpretation

A page is important if important pages point to it.

Recursive scoring idea

Importance flows along incoming links.

This is a fixed-point problem

Scores depend on other scores.

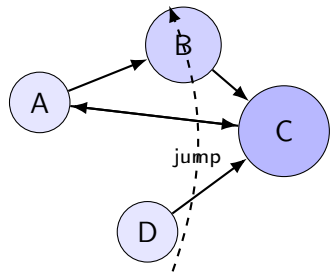
PageRank as Random Surfer Model

Mental picture

A surfer repeatedly follows outgoing links, with occasional random jumps.

Interpretation

The stationary probability of being at a page becomes its rank score.



Centrality Measures

Big question

What does “important node” mean?

Answer

There is no single notion of importance; different centrality measures capture different roles.

Degree Centrality

Definition

Count the number of incident edges.

Interpretation

Nodes with many direct connections are locally influential or highly active.

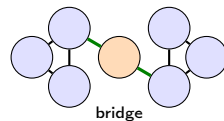
Closeness and Betweenness Centrality

Closeness

Nodes with short average distance to others are globally well positioned.

Betweenness

Nodes that lie on many shortest paths act as bridges or bottlenecks.



Discussion Exercise: Which Node Is Most Important?

Prompt

In a transportation network, which node should be considered the most important?

Possible answers depend on the metric

- highest degree: many direct links
- highest closeness: fast access to the whole system
- highest betweenness: critical transfer hub
- highest PageRank: receives traffic from already important places

Teaching goal

Students should stop thinking of “importance” as a single objective fact.

A* Search: Why Another Path Algorithm?

Motivation

In pathfinding, we often care about one source-target pair, not all distances from the source.

Goal

Use extra problem knowledge to search more directly toward the target.

A* Search: Scoring Rule

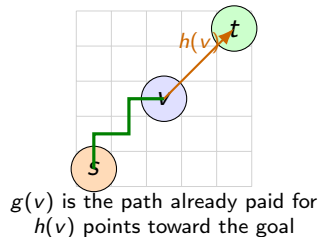
Priority key

For a candidate vertex v , use

$$f(v) = g(v) + h(v)$$

where

- $g(v)$: cost from source to v
- $h(v)$: heuristic estimate from v to target



A* Search: Connection to Dijkstra

Special case

If

$$h(v) = 0 \quad \text{for all } v,$$

then A* becomes Dijkstra.

Interpretation

A* is Dijkstra plus guidance.

When A* Works Well

Good heuristic

The heuristic should be informative but not overoptimistic in the wrong direction.

Typical example

Euclidean distance to the goal in map routing.

Use case

Games, navigation, robotics, route planning.

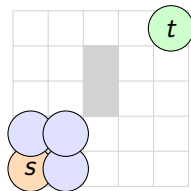
Worked Example: A* on a Grid

Task

Suppose movement cost is 1 per step and $h(v)$ is Manhattan distance to the goal.

Ask

Which frontier node has the smallest $f = g + h$ score after the first few expansions?



The Nearest Neighbor Problem

Task

Given a query point, find the closest stored point under a distance measure.

Applications

Recommendation, image retrieval, anomaly detection, classification, vector search.

k-NN Classification

Basic rule

Find the k closest training points and let them vote on the label.

Algorithmic challenge

Training is cheap, but prediction can be expensive if we scan all stored points.

Why Brute Force Becomes Expensive

If there are N points in d dimensions

One exact query by full scan typically costs

$$\Theta(Nd).$$

Problem

This can be too slow when N is very large or queries are frequent.

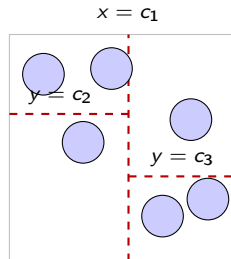
k-d Trees

Core idea

Recursively split space by axis-aligned cuts.

Node meaning

Each internal node stores one coordinate split and partitions the point set into two regions.



How k-d Tree Search Prunes Work

Main rule

First follow the branch that contains the query point.

Then

Only explore the other branch if its region could still contain a closer point than the best one found so far.

When k-d Trees Work Well

Best setting

Low or moderate dimension, reasonably structured data.

Warning

In high dimensions, pruning often weakens and brute force may become competitive again.

Ball Trees

Alternative partition idea

Instead of axis-aligned boxes, organize points into nested metric balls.

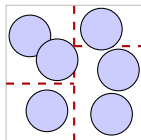
Why useful

Ball trees can adapt better than k-d trees when Euclidean coordinates are not the whole story.

k-d Trees vs Ball Trees

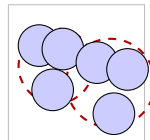
k-d tree

Axis-aligned partition.



Ball tree

Metric-space partition by radius.



Practical note

Both are exact search structures, but their usefulness depends on geometry and dimension.

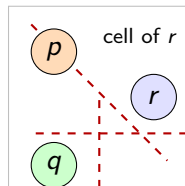
Voronoi Diagrams

Geometric view of nearest neighbors

Partition the plane into regions where each region contains all points closest to one site.

Interpretation

If the query lies inside the Voronoi cell of site p , then p is the nearest neighbor.



Spatial Search Exercise

Prompt

You have a million 2D points and many nearest-neighbor queries.

Discuss in pairs

Which method would you try first?

- brute-force scan
- k-d tree
- ball tree

What changes if the points live in 200 dimensions instead?

Target insight

The geometry of the data can matter as much as the asymptotic complexity written on the slide.

Which Graph or Spatial Algorithm Should I Use?

Problem	Typical tool
unweighted shortest paths	BFS
graph exploration / components	DFS
weighted shortest paths	Dijkstra
all-pairs shortest paths	Floyd-Warshall
minimum total network cost	Kruskal / MST
one target with heuristic guidance	A*
nearest-neighbor search	k-d tree / ball tree / scan

Final Checkpoint Questions

- 1 Why is BFS correct for shortest paths only in unweighted graphs?
- 2 Why does Dijkstra require nonnegative edge weights?
- 3 When is Floyd-Warshall a better fit than repeated single-source search?
- 4 Why is an MST not the same thing as a shortest-path tree?
- 5 When do k-d trees start to lose their advantage?

The Bigger Picture

- Graph algorithms solve path, flow, and connectivity questions.
- Network analysis asks which vertices matter most.
- Spatial search asks how to organize geometry for fast queries.
- The right model and the right data structure are as important as the algorithm itself.

What to Remember

- BFS and DFS are the two foundational graph traversals.
- Dijkstra and Floyd-Warshall solve different shortest-path settings.
- Kruskal connects a network cheaply, not quickly.
- PageRank and centrality scores turn graphs into analytical tools.
- A* and spatial trees use structure to cut search time.

Next Time

Coming next

We move from efficient algorithms to the boundary of efficient computation: lower bounds, reductions, NP-completeness, and approximation.